NC State University

Department of Electrical and Computer Engineering

ECE 463/563 (Prof. Rotenberg)

Project #1: Cache Design, Memory Hierarchy Design

REPORT TEMPLATE (Version 2.0)

by

RUSHIRAJ CHAITANYAKUMAR SHETH

NCSU Honor Pledge: "I have neither given nor received unauthorized aid on this project."

Student's electronic signature: _Rushiraj Chaitanyakumar Sheth_______________
(sign by typing your name)

Course number: ___563____
(463 or 563 ?)

Report template for Project #1.

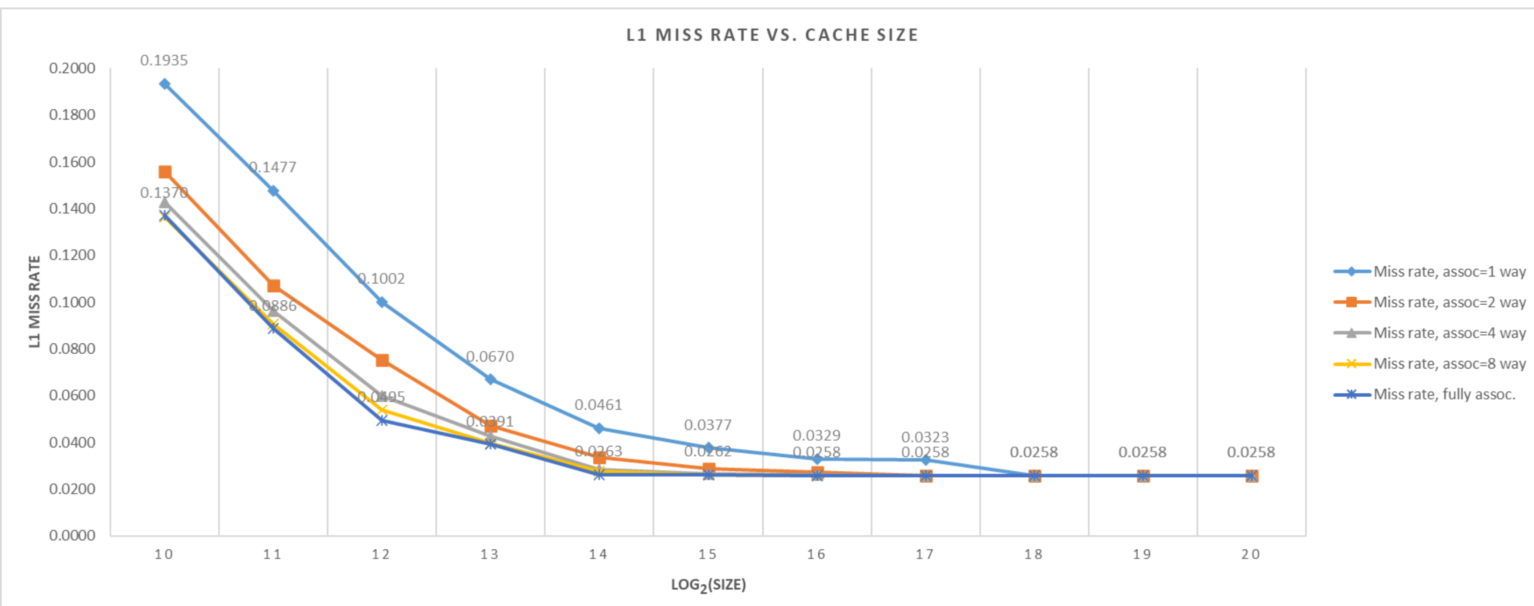
1. L1 cache exploration: SIZE and ASSOC

GRAPH #1 (***total number of simulations: 55***)

For this experiment:

- Benchmark trace: gcc_trace.txt
- L1 cache: SIZE is varied, ASSOC is varied, BLOCKSIZE = 32.
- L2 cache: None.
- Prefetching: None.

Plot L1 miss rate on the y-axis versus $\log_2$(SIZE) on the x-axis, for eleven different cache sizes: SIZE = 1KB, 2KB, … , 1MB, in powers-of-two. (That is, $\log_2$(SIZE) = 10, 11, …, 20.) The graph should contain five separate curves (*i.e.*, lines connecting points), one for each of the following associativities: direct-mapped, 2-way set-associative, 4-way set-associative, 8-way set-associative, and fully-associative. All points for direct-mapped caches should be connected with a line, all points for 2-way set-associative caches should be connected with a line, *etc*.



Answer the following questions:

1. For a given associativity, how does increasing cache size affect miss rate?

(write answer here)
For a given associativity, the miss rate goes on decreasing drastically till cache size is 16KB(2^14). Then there are diminishing changes i.e. the decrease in miss rate is not much and increasing the cache size further does not show significant decrease in miss rate. From cache size as 256KB, the miss rate becomes constant for all types of associativity, because the cache becomes large enough that conflict and capacity misses are removed and only compulsory misses are prevailing.

2. For a given cache size, how does increasing associativity affect miss rate?

(write answer here)
Increase in associativity for a same cache size, results in reduced miss rates. As we can see, for a given small cache size (1KB to 16KB), the miss rates are decreasing from a 1 way assoc. to fully assoc. cache. For a given cache size, the increase in associativity leads to decrease in conflict misses and hence, the overall miss rate reduces.
However, beyond the size of 16KB the decrease in miss rate is not significant(for different assoc. for a particular cache size) and eventually becomes constant.

3. Estimate the *compulsory miss rate* from the graph and briefly explain how you arrived at this estimate.

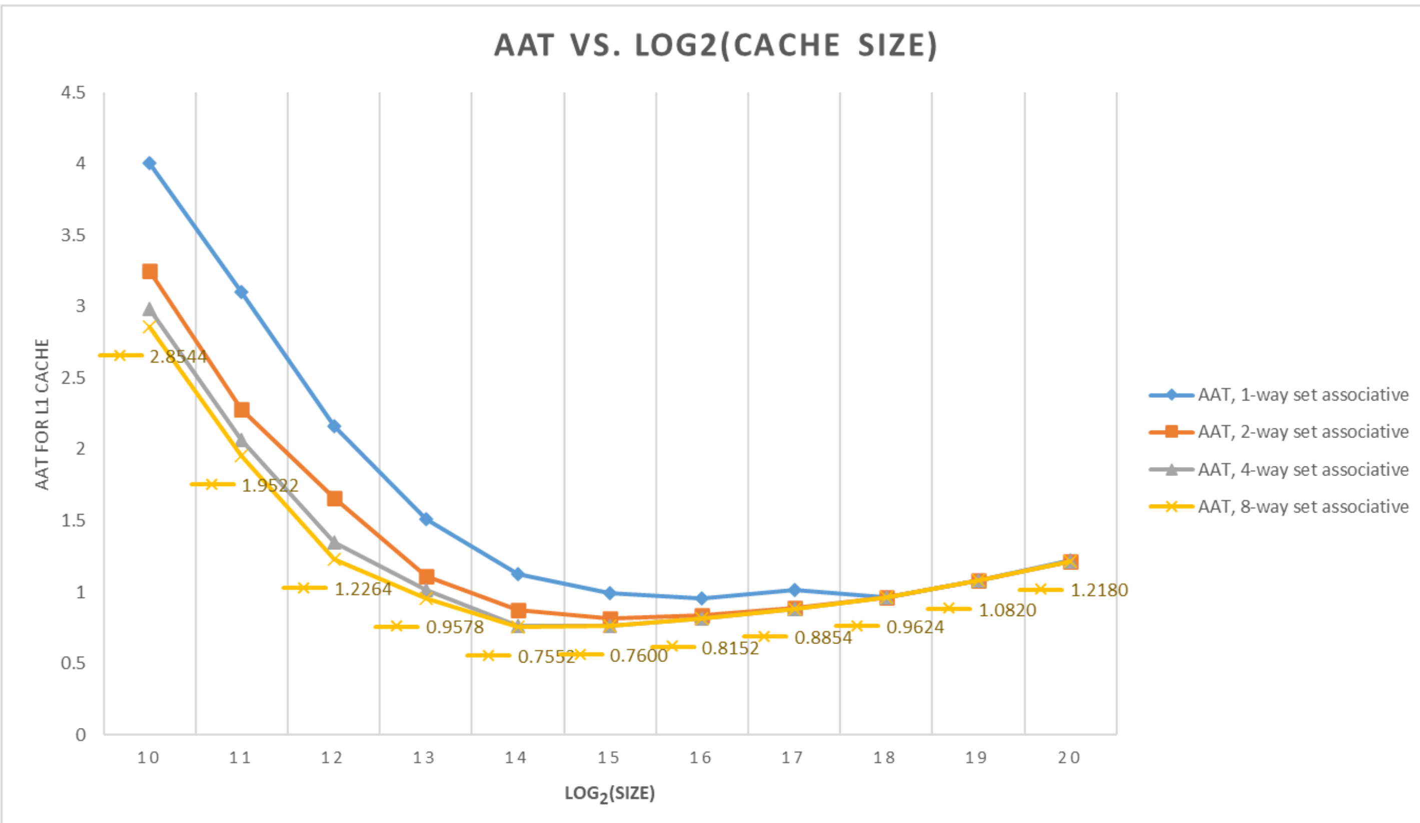
compulsory miss rate = _0.0258__
How I arrived at this estimate: _as seen in the graph, the cache size is large enough(here 1MB), that only compulsory misses exists. The graph of miss rate vs. log2(cache size), provides the information about compulsory misses in the region of “diminishing returns”._

GRAPH #2 (***no additional simulations with respect to GRAPH #1***)

Same as GRAPH #1, except make the following changes:

- The y-axis should be AAT instead of L1 miss rate.
- Do NOT include fully-associative cache configurations (power-hungry, impractical cost for implementing LRU). Thus, GRAPH #2 should contain four (not five) separate curves: direct-mapped, 2-way set-associative, 4-way set-associative, 8-way set-associative.

<< INSERT GRAPH #2 HERE >>



Answer the following question:

1. For a memory hierarchy with only an L1 cache and BLOCKSIZE = 32, which configuration yields the best (*i.e.*, lowest) AAT and what is that AAT?

Direct-mapped or set-associative cache configuration that yields the lowest AAT: 16 KB, 8-way set-assoc. AAT for this configuration: ________0.7552____________

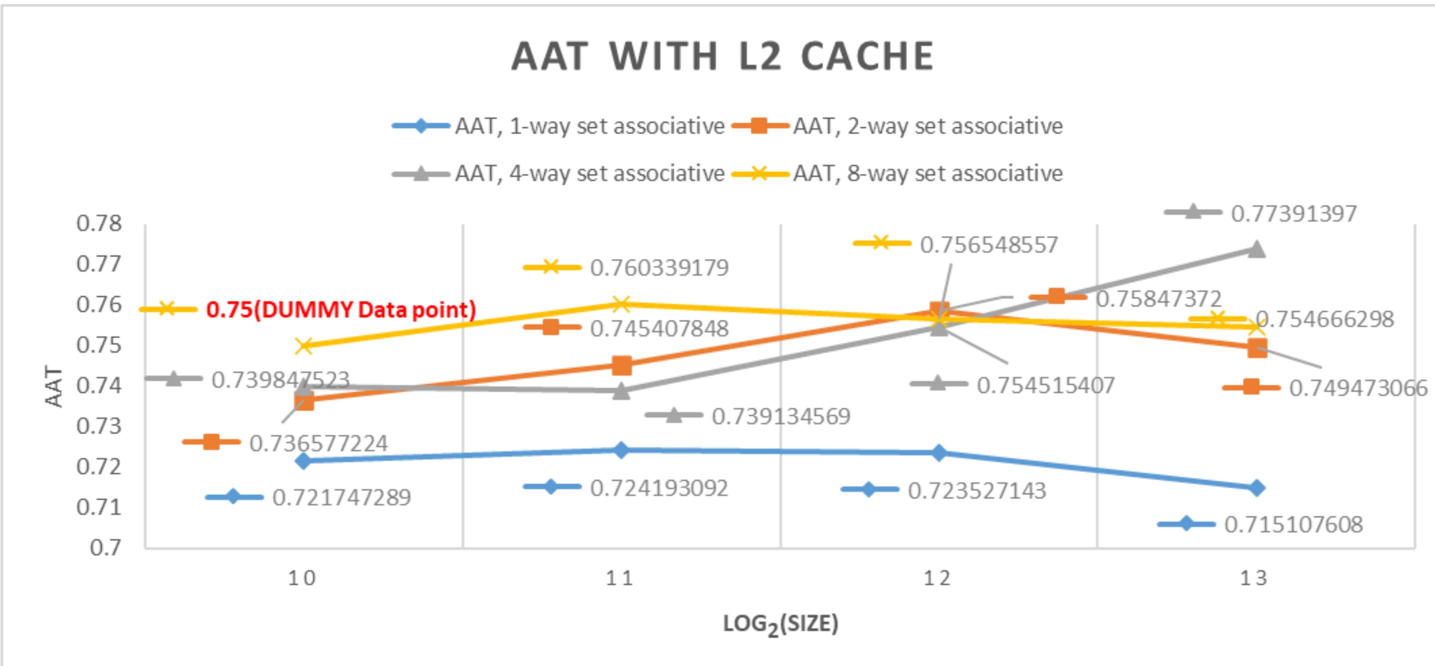
Report template for Project #1.

<u>GRAPH #3</u> (***total number of simulations: 16***)

Same as GRAPH #2, except make the following changes:

- Add the following L2 cache to the memory hierarchy: 16KB, 8-way set-associative, same block size as L1 cache.
- Vary the L1 cache size only between 1KB and 8KB (since L2 cache is 16KB). (*And as with GRAPH #2, GRAPH #3 should contain four (not five) separate curves for L1 associativity: direct-mapped, 2-way set-associative, 4-way set-associative, 8-way set-associative.*)

<< INSERT GRAPH #3 HERE >>



Answer the following questions:

1. With the L2 cache added to the system, which L1 cache configuration yields the best (*i.e.*, lowest) AAT and what is that AAT?

L1 configuration that yields the lowest AAT with 16KB 8-way L2 added: <u>8 KB, direct-mapped</u> AAT for this configuration: _0.7151_____________________

2. How does the lowest AAT with L2 cache (GRAPH #3) compare with the lowest AAT without L2 cache (GRAPH #2)?

The lowest AAT with L2 cache is <u>less than</u> the lowest AAT without L2 cache.

2. L1 cache exploration: SIZE and BLOCKSIZE

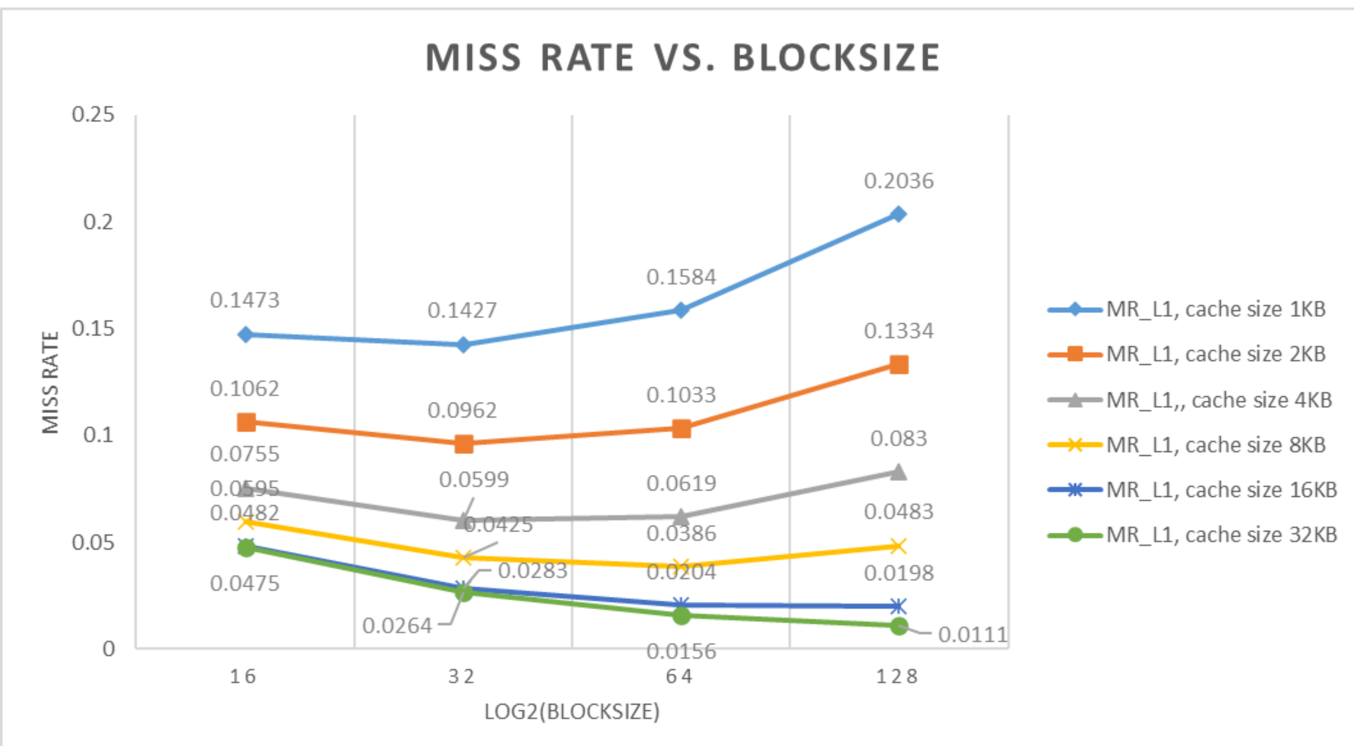
<u>GRAPH #4</u> (***total number of simulations: 24***)

For this experiment:

- Benchmark trace: gcc_trace.txt
- L1 cache: SIZE is varied, BLOCKSIZE is varied, ASSOC = 4.
- L2 cache: None.
- Prefetching: None

Plot L1 miss rate on the y-axis versus $\log_2$(BLOCKSIZE) on the x-axis, for four different block sizes: BLOCKSIZE = 16, 32, 64, and 128. (That is, $\log_2$(BLOCKSIZE) = 4, 5, 6, and 7.) The graph should contain six separate curves (*i.e.*, lines connecting points), one for each of the following L1 cache sizes: SIZE = 1KB, 2KB, …, 32KB, in powers-of-two. All points for SIZE = 1KB should be connected with a line, all points for SIZE = 2KB should be connected with a line, *etc*.

<< INSERT GRAPH #4 HERE >>



Answer the following questions:

1. Do smaller caches prefer smaller or larger block sizes?

Smaller caches prefer smaller block sizes. For example, the smallest cache considered in Graph #4 (1KB) achieves its lowest miss rate with a block size of _16___ B.

2. Do larger caches prefer smaller or larger block sizes?

Larger caches prefer larger block sizes. For example, the largest cache considered in Graph #4 (32KB) achieves its lowest miss rate with a block size of __128__ B.

3. As block size is increased from 16 to 128, is the tension between *exploiting more spatial locality* and *cache pollution* evident in the graph? Explain.

Yes, the tension between *exploiting more spatial locality* and *cache pollution* is evident in the graph.

For example, consider the smallest (1KB) cache in Graph #4. Increasing block size from _16 B to 32 B is helpful (reduces miss rate) due to exploiting more spatial locality. But then increasing block size further, from _32_ B to _128_ B, is not helpful (increases miss rate) due to cache pollution having greater effect.

3. L1 + L2 co-exploration

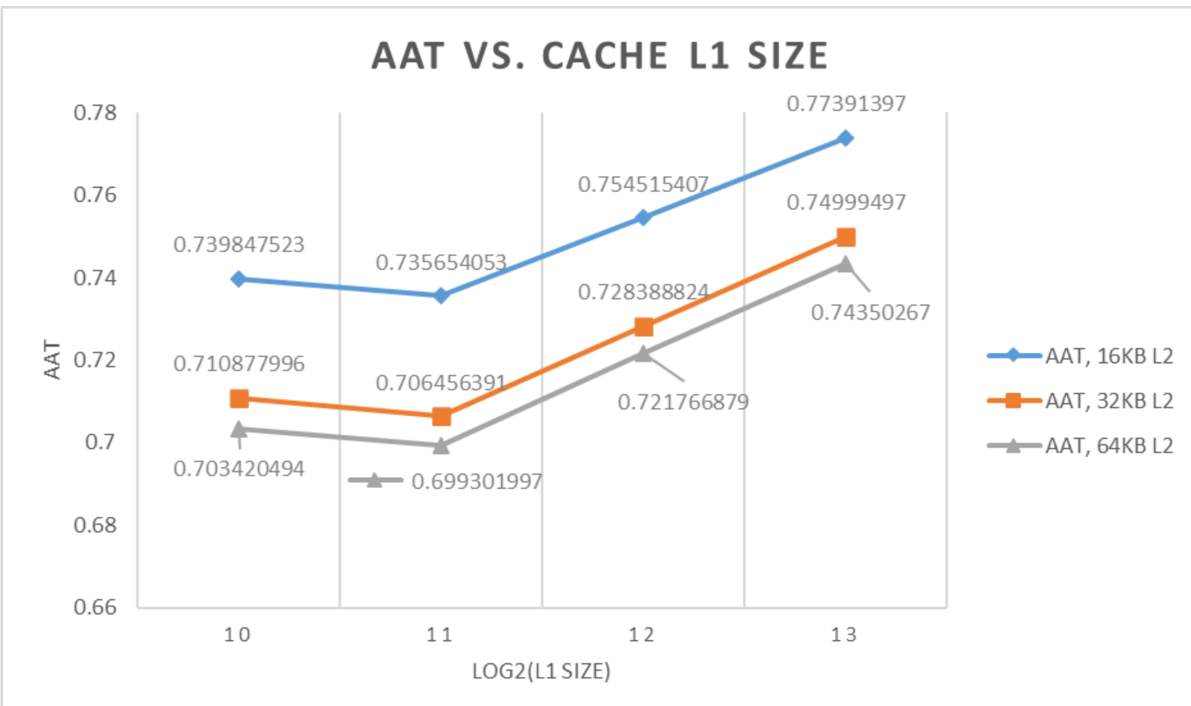
<u>GRAPH #5</u> (***total number of simulations: 12***)

For this experiment:

- Benchmark trace: gcc_trace.txt
- L1 cache: SIZE is varied, BLOCKSIZE = 32, ASSOC = 4.
- L2 cache: SIZE is varied, BLOCKSIZE = 32, ASSOC = 8.
- Prefetching: None.

Plot AAT on the y-axis versus $\log_2$(L1 SIZE) on the x-axis, for four different L1 cache sizes: L1 SIZE = 1KB, 2KB, 4KB, 8KB. (That is, $\log_2$(L1 SIZE) = 10, 11, 12, 13.) The graph should contain three separate curves (*i.e.*, lines connecting points), one for each of the following L2 cache sizes: 16KB, 32KB, 64KB. All points for the 16KB L2 cache should be connected with a line, all points for the 32KB L2 cache should be connected with a line, *etc*.

<< INSERT GRAPH #5 HERE >>



Answer the following question:

1. Which memory hierarchy configuration in Graph #5 yields the best (*i.e.*, lowest) AAT and what is that AAT?

Configuration that yields the lowest AAT: _L1 Cache: 2KB, Assoc=4 and L2 Cache: 64KB, Assoc=8. _ Lowest AAT: _____0.6993_________________

4. Stream buffers study (ECE 563 students only)

<u>TABLE #1</u> (***total number of simulations: 5***)

For this experiment:

- Microbenchmark: stream_trace.txt
- L1 cache: SIZE = 1KB, ASSOC = 1, BLOCKSIZE = 16.
- L2 cache: None.
- PREF_N (number of stream buffers): 0 (pref. disabled), 1, 2, 3, 4
- PREF_M (number of blocks in each stream buffer): 4

The trace "stream_trace.txt" was generated from the loads and stores in the loop of interest of the following microbenchmark:

```
#define SIZE 1000

uint32_t a[SIZE];
uint32_t b[SIZE];
uint32_t c[SIZE];

int main(int argc, char *argv[]) {
…
   // LOOP OF INTEREST
   for (int i = 0; i < SIZE; i++)
      c[i] = a[i] + b[i];   // per iteration: 2 loads (a[i], b[i]) and 1 store (c[i] = …)
…
}
```

Fill in the following table and answer the following questions:

PREF_N, PREF_M	L1 miss rate
0,0 (pref. disabled)	0.2500
1,4	0.2500
2,4	0.2500
3,4	0.0010
4,4	0.0010

1. For this streaming microbenchmark, with prefetching disabled, do L1 cache size and/or associativity affect the L1 miss rate (feel free to simulate L1 configurations besides the one used for the table)? Why or why not?

With prefetching disabled, L1 cache size and/or associativity <u>do not</u> affect L1 miss rate (for this streaming microbenchmark).

The reason: _ the miss rate corresponds to compulsory misses._ .

2. For this streaming microbenchmark, what is the L1 miss rate with prefetching disabled? Why is it that value, *i.e.*, what is causing it to be that value? Hint: each element of arrays a, b, and c, is 4 bytes (uint32_t).

The L1 miss rate with prefetching disabled is ______0.2500_____, because _size of each array is 4000 bytes(4 bytes int x1000) and for 3 arrays it is 12000 bytes. Total no. of sets

in cache is 1024. Since, cache has blocksize of 16B/block, no. of sets needed to hold all the elements of array = 12000 bytes/(16 bytes per block or set) = 750 no. of sets.
Also, total L1 reads+writes = 3000 and L1 readmisses + writemisses = 750 for direct-mapped cache. Hence, the total misses are equal to the required no. of sets. So, these misses are compulsory misses and miss rate = 750/3000 = 0.25 .

3. For this streaming microbenchmark, with prefetching disabled, what would the L1 miss rate be if you doubled the block size from 16B to 32B? (hypothesize what it will be and then check your hypothesis with a simulation)

The L1 miss rate with prefetching disabled and a block size of 32B is ___0.1260__, because _available cache blocks are 512(total sets) and cache blocks needed to store the elements = 12000 bytes/32 bytes per block = 375. Since the total number of blocks needed (375) is less than the available blocks (512), we would expect low cache misses. The L1 read misses are 252 and write misses are 126. L1 miss rate = (252+126)/3000 = 0.1260__.

4. With prefetching enabled, what is the minimum number of stream buffers required to have any effect on L1 miss rate? What is the effect on L1 miss rate when this many stream buffers are used: specifically, is it a modest effect or huge effect? Why are this many stream buffers required? Why is using fewer stream buffers futile? Why is using more stream buffers wasteful?

Minimum number of stream buffers needed to have any effect on L1 miss rate: __3___

With this many stream buffers, the effect on L1 miss rate is << huge >>. Specifically, the L1 miss rate is nearly __0.001___. We only miss on the first elements of ______arrays___________ (hence a total of _3_ misses).

This many stream buffers are required because __we need to prefetch for each of the arrays i.e.a[], b[], c[]_______.

Using fewer stream buffers is futile because we cannot store prefetches for each of the 3 arrays in <3 stream buffers. If stream buffers are <3, it will be able to hold prefetches for any 2 (or 1, if only 1 stream buffer) arrays and on access/demand miss of 3rd array one of the buffer will be flushed and prefetched again. Hence, the memory traffic is increasing and miss rate is increasing._

Using more stream buffers is wasteful because _since we have 3 different arrays, the 1st access to each array will cause a demand miss and the next blocks' stride can be prefetched in 3 different stream buffers for each array. More than 3 buffers will not be useful as we have only 3 entities to prefetch and hence, miss rate will also not be affected. _______.